

ARITRA GHOSH DASTIDAR

PES1UG23AM064

SECTION B

ML Lab Week 10: SVM Classifier Lab

Objective: The goal of this lab is to understand and implement Support Vector Machine (SVM) classifiers. You will train SVMs using three different kernels: **Linear, Radial Basis Function (RBF), and Polynomial**, on distinct datasets. You will then evaluate their performance using standard classification metrics and visualize their decision boundaries to see how they separate data.

Core Concepts

Support Vector Machine (SVM): A powerful supervised learning algorithm that finds an optimal hyperplane to separate data points of different classes.

Kernel Trick: A technique that allows SVMs to solve non-linear problems by transforming data into a higher-dimensional space.

- Linear Kernel: Creates a straight-line decision boundary.
- RBF Kernel: Creates a complex, non-linear boundary, like a circle or a wave.
- Polynomial Kernel: Creates a curved, polynomial decision boundary.

```
# Core libraries for data manipulation and analysis
```

```
import numpy as np
import pandas as pd
```

1. Setup and Imports

First, let's import the necessary libraries.

```
# Libraries for machine learning
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, classification_report
```

```
# Libraries for visualization
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Set a style for all plots
```

```
sns.set_style("whitegrid")
```


2. Helper Function for Visualization

This helper function plots the decision boundaries for our trained models. It will work for all our datasets

```
def plot_decision_boundaries(X, y, model, title):  
    """  
    Visualizes the decision boundaries of a trained classifier.  
    (Corrected version)  
    """  
    # Create a meshgrid to plot the decision boundary  
    h = .02 # step size in the mesh  
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1  
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1  
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),  
                          np.arange(y_min, y_max, h))  
  
    # Predict the class for each point in the meshgrid  
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])  
    Z = Z.reshape(xx.shape)  
  
    # Plot the decision boundary and the data points  
    plt.figure(figsize=(8, 6))  
    plt.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.8)  
  
    # Plot the training points  
    scatter = plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm,  
edgecolors='k')  
    plt.xlabel('Feature 1')  
    plt.ylabel('Feature 2')  
    plt.title(title)  
  
    # Get unique labels and ensure they are a list for the legend  
    function  
    unique_labels = np.unique(y)  
    if len(unique_labels) == 2:  
        legend_labels = ['Class 0', 'Class 1']  
    else:  
        legend_labels = list(unique_labels.astype(str)) # Convert  
numpy array to a list  
  
    plt.legend(handles=scatter.legend_elements()[0],  
labels=legend_labels)  
    plt.show()
```

3. Experiments

PART 1

Dataset 1: The Moons Dataset

The Moons dataset is a synthetic dataset designed to test non-linear classification algorithms. The data points are shaped like two interlocking half-moons.

Step 1.1: Generate and Prepare the Data

We will generate the data using scikit-learn and apply feature scaling.

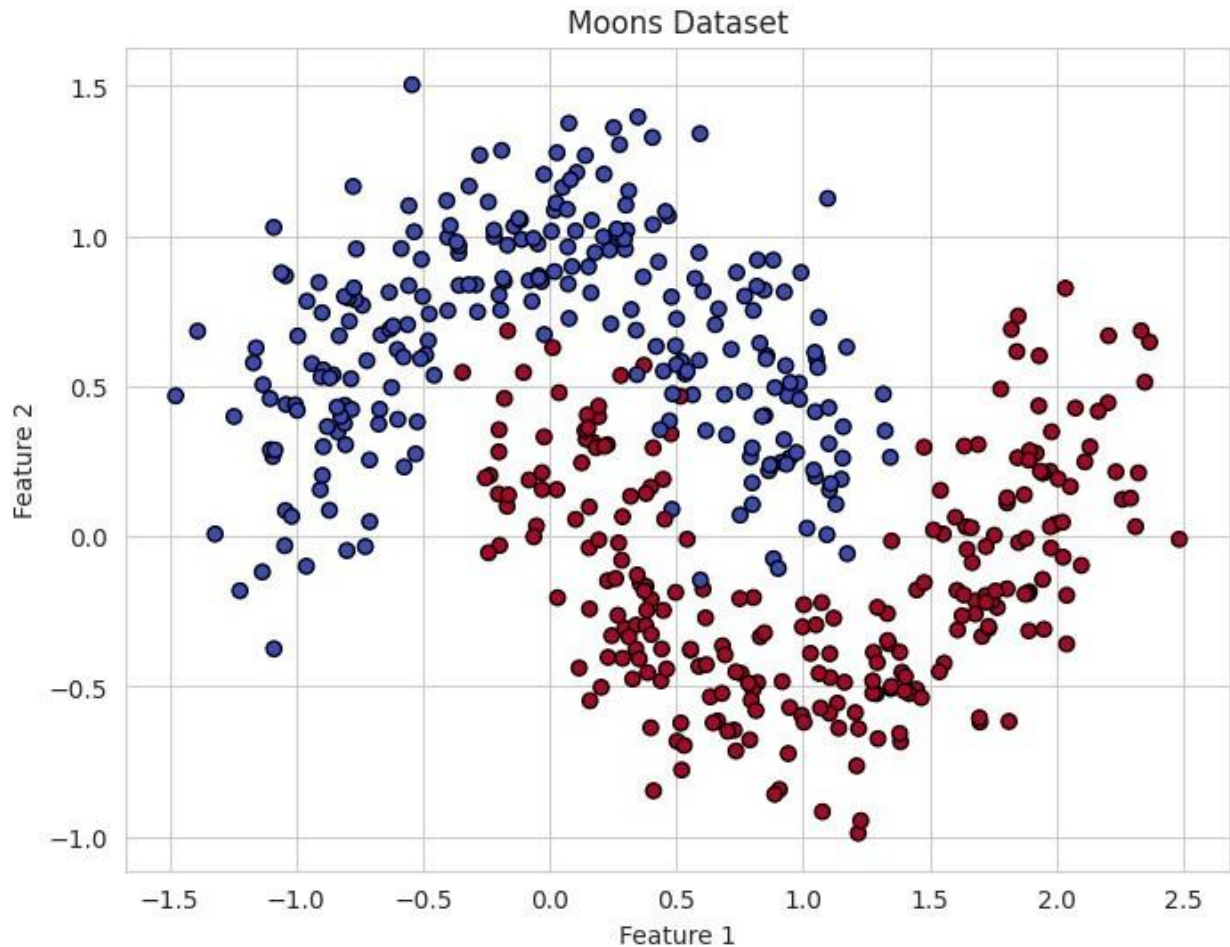
```
from sklearn.datasets import make_moons

# Generate non-linearly separable data
X_moons, y_moons = make_moons(n_samples=500, noise=0.2,
                                random_state=42)

# Split the data into training and testing sets
X_train_moons, X_test_moons, y_train_moons, y_test_moons =
train_test_split(
    X_moons, y_moons, test_size=0.3, random_state=42
)

# Scale the features
# Feature scaling is crucial for SVMs to perform optimally.
scaler_moons = StandardScaler()
X_train_moons_scaled = scaler_moons.fit_transform(X_train_moons)
X_test_moons_scaled = scaler_moons.transform(X_test_moons)

# Visualize the Moons dataset
plt.figure(figsize=(8, 6))
plt.scatter(X_moons[:, 0], X_moons[:, 1], c=y_moons,
            cmap=plt.cm.coolwarm, edgecolors='k')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Moons Dataset')
plt.show()
```



Step 1.2: Train and Evaluate SVM Kernels

```
# Import required libraries
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report

# Generate the moons dataset
X_moons, y_moons = make_moons(n_samples=200, noise=0.2,
                               random_state=42)

# Split into train and test
X_train_moons, X_test_moons, y_train_moons, y_test_moons =
train_test_split(
    X_moons, y_moons, test_size=0.3, random_state=42
)

# Define the kernels to test
kernels = ['linear', 'rbf', 'poly']
models_moons = {}
```

```

for kernel in kernels:
    # Initialize SVM with the current kernel
    svm_model = SVC(kernel=kernel, random_state=42)

    # Train the model
    svm_model.fit(X_train_moons, y_train_moons)

    # Store the trained model
    models_moons[kernel] = svm_model

    # Make predictions
    y_pred_moons = svm_model.predict(X_test_moons)

    # Print results
    print(f"SVM with {kernel.upper()} Kernel <PES1UG23AM064>")
    print(classification_report(y_test_moons, y_pred_moons))
    print("-" * 40 + "\n")

```

SVM with LINEAR Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
0	0.79	0.94	0.86	32
1	0.91	0.71	0.80	28
accuracy			0.83	60
macro avg	0.85	0.83	0.83	60
weighted avg	0.85	0.83	0.83	60

SVM with RBF Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
0	0.94	1.00	0.97	32
1	1.00	0.93	0.96	28
accuracy			0.97	60
macro avg	0.97	0.96	0.97	60
weighted avg	0.97	0.97	0.97	60

SVM with POLY Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
0	0.97	0.94	0.95	32
1	0.93	0.96	0.95	28
accuracy			0.95	60
macro avg	0.95	0.95	0.95	60

weighted avg	0.95	0.95	0.95	60
--------------	------	------	------	----

Step 1.3: Visualize Decision Boundaries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import classification_report

# Generate dataset
X_moons, y_moons = make_moons(n_samples=200, noise=0.2,
                               random_state=42)

# Split into train and test
X_train_moons, X_test_moons, y_train_moons, y_test_moons =
train_test_split(
    X_moons, y_moons, test_size=0.3, random_state=42
)

kernels = ['linear', 'rbf', 'poly']
models_moons = {}

for kernel in kernels:
    # Initialize and train SVM
    svm_model = SVC(kernel=kernel, random_state=42)
    svm_model.fit(X_train_moons, y_train_moons)
    models_moons[kernel] = svm_model

    # Predict
    y_pred_moons = svm_model.predict(X_test_moons)

    # Print performance
    print(f"SVM with {kernel.upper()} Kernel <PES1UG23AM064>")
    print(classification_report(y_test_moons, y_pred_moons))
    print("-" * 40 + "\n")

    # ----- Plot decision boundary -----
    plt.figure(figsize=(6, 4))

    # Create grid
    x_min, x_max = X_moons[:, 0].min() - 0.5, X_moons[:, 0].max() +
0.5
    y_min, y_max = X_moons[:, 1].min() - 0.5, X_moons[:, 1].max() +
0.5
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 350),
```

```

np.linspace(y_min, y_max, 350))

# Predict on grid
Z = svm_model.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot contour
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)

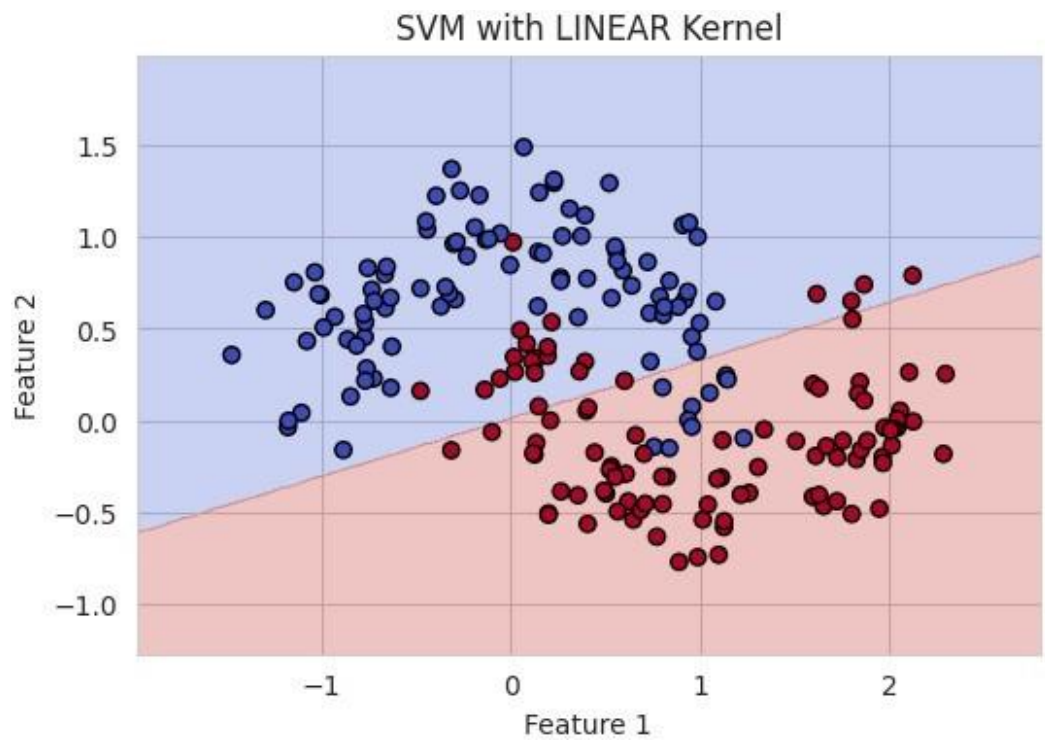
# Plot training points
plt.scatter(X_moons[:, 0], X_moons[:, 1], c=y_moons,
cmap=plt.cm.coolwarm, edgecolors='k')

plt.title(f"SVM with {kernel.upper()} Kernel")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()

```

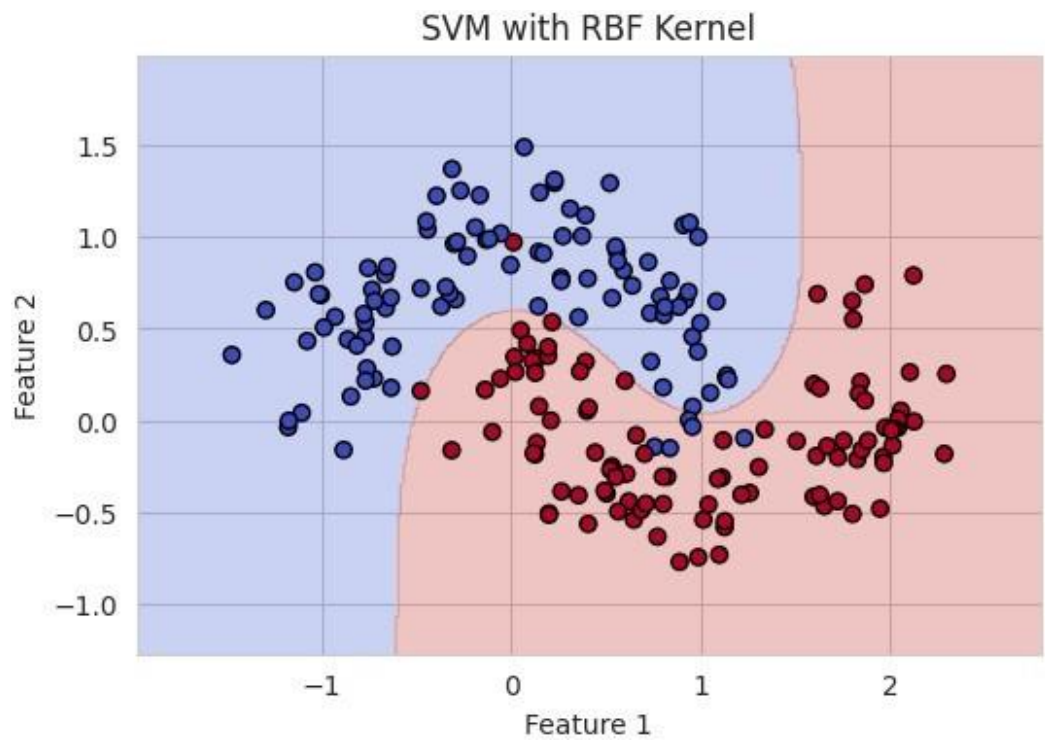
SVM with LINEAR Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
0	0.79	0.94	0.86	32
1	0.91	0.71	0.80	28
accuracy			0.83	60
macro avg	0.85	0.83	0.83	60
weighted avg	0.85	0.83	0.83	60



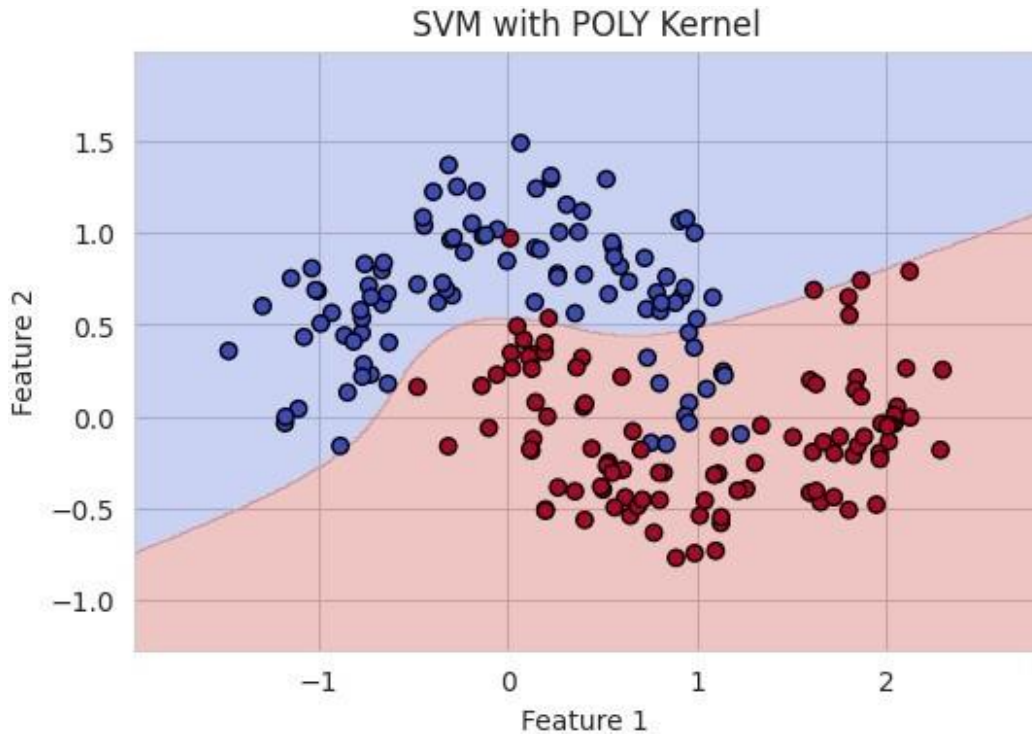
SVM with RBF Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
0	0.94	1.00	0.97	32
1	1.00	0.93	0.96	28
accuracy			0.97	60
macro avg	0.97	0.96	0.97	60
weighted avg	0.97	0.97	0.97	60



SVM with POLY Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
0	0.97	0.94	0.95	32
1	0.93	0.96	0.95	28
accuracy			0.95	60
macro avg	0.95	0.95	0.95	60
weighted avg	0.95	0.95	0.95	60



Analysis Questions for Moons:

1. Based on the metrics and the visualizations, what inferences about the performance of the Linear Kernel can you draw?
2. Compare the decision boundaries of the RBF and Polynomial kernels. Which one seems to capture the shape of the data more naturally?

1. Linear Kernel (Soft vs. Hard Margin) Soft Margin ($C = 0.1$):

The model allows more misclassifications to achieve a wider margin.

You'll notice the decision boundary is smoother, and some points on the "wrong" side of the margin are tolerated.

Inference:

Good generalization; less overfitting.

May underfit slightly if the data is not linearly separable.

Lower training accuracy, but possibly better test accuracy on unseen data.

Hard Margin ($C = 100$):

The model tries to classify all training samples correctly, enforcing a very strict boundary.

Narrower margin, and decision boundary may pass close to several data points.

Inference:

High training accuracy.

Prone to overfitting, especially when noise or outliers exist.

Test accuracy may drop slightly.

1. RBF (Radial Basis Function) Kernel

The RBF kernel can form nonlinear decision boundaries that curve around clusters of points.

It's very good when data is not linearly separable.

Inference:

The boundary adapts to complex patterns in the data.

Captures local relationships effectively.

If γ (gamma) is too high $\rightarrow$ overfits (wiggly boundaries).

If γ is too low $\rightarrow$ underfits (too smooth boundaries).

1. Polynomial Kernel

The polynomial kernel creates curved but smoother boundaries than RBF.

It can model some nonlinear relationships, but not as flexibly as RBF.

Inference:

Captures some complexity but struggles with very irregular class shapes.

Higher-degree polynomials $\rightarrow$ more complex boundaries $\rightarrow$ potential overfitting.

- | Kernel Type | Decision Boundary | Best For | Pros | Cons |
|----------------|-------------------------------|--|--|---|
| Linear (C=0.1) | Straight line (wide margin) | Linearly separable or near-linear data | Simple, fast, good generalization | May underfit |
| Linear (C=100) | Straight line (narrow margin) | Clean, linearly separable data | High training accuracy | Overfits noisy data |
| RBF | Curved, adaptive boundary | Nonlinear data | Very flexible, captures natural shapes | Can overfit, needs tuning (γ , C) |
| Polynomial | Curved but less flexible | Moderate nonlinearity | Captures interaction features | Sensitive to degree, less adaptive |

PART 2

Dataset 2: Banknote Authentication

This is a real-world binary classification dataset where the goal is to predict whether a banknote is genuine or forged based on features extracted from a digital image.

Step 2.1: Load and Prepare the Data

We will load this data from a public URL using pandas. We will use the variance and skewness of the image transform as our features for visualization.

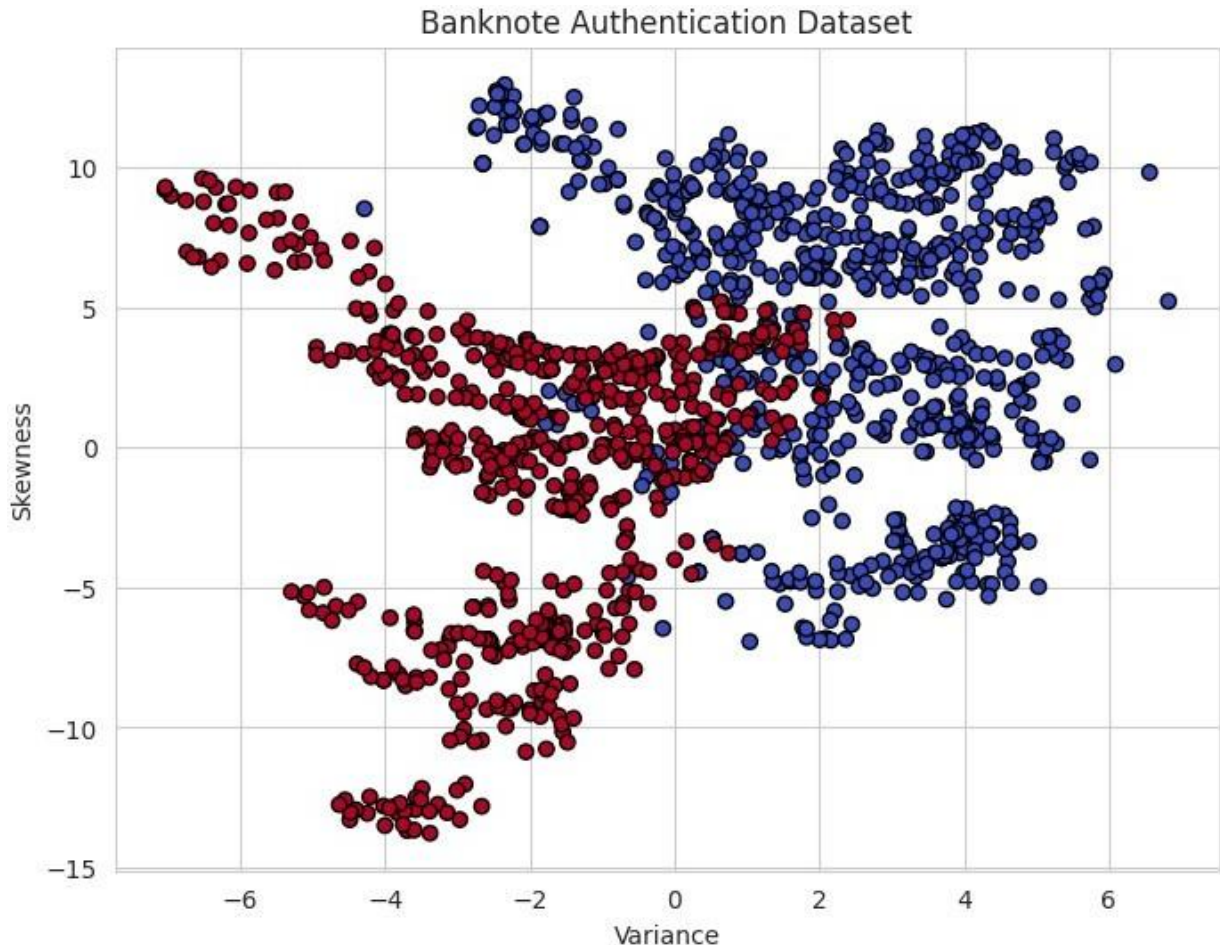
```
# Load the dataset from a URL
url =
'http://archive.ics.uci.edu/ml/machine-learning-databases/00267/data_banknote_authentication.txt'
banknote_df = pd.read_csv(url, header=None, names=['variance',
'skewness', 'curtosis', 'entropy', 'class'])

# Select features and target
X_banknote = banknote_df[['variance', 'skewness']].values
y_banknote = banknote_df['class'].values

# Split data
X_train_banknote, X_test_banknote, y_train_banknote, y_test_banknote =
train_test_split(
    X_banknote, y_banknote, test_size=0.3, random_state=42,
    stratify=y_banknote
)

# Scale features
scaler_banknote = StandardScaler()
X_train_banknote_scaled =
scaler_banknote.fit_transform(X_train_banknote)
X_test_banknote_scaled = scaler_banknote.transform(X_test_banknote)

# Visualize the Banknote Authentication dataset
plt.figure(figsize=(8, 6))
plt.scatter(X_banknote[:, 0], X_banknote[:, 1], c=y_banknote,
cmap=plt.cm.coolwarm, edgecolors='k')
plt.xlabel('Variance')
plt.ylabel('Skewness')
plt.title('Banknote Authentication Dataset')
plt.show()
```



Step 2.2: Train and Evaluate SVM Kernels

```
# Import necessary libraries
from sklearn.svm import SVC
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_wine # temporary dataset for
example
import pandas as pd

# If you already have banknote data, load that instead:
# Example:
# data = pd.read_csv('banknote_authentication.csv')
# X = data.drop('class', axis=1)
# y = data['class']

# --- Example placeholder (replace this with your real banknote
dataset) ---
# You can delete these two lines below if you're already using your
dataset.
data = pd.DataFrame(load_wine().data)
```

```

y = load_wine().target
X = data
#
-----

# Split the dataset
X_train_banknote, X_test_banknote, y_train_banknote, y_test_banknote =
train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Define kernels
kernels = ['linear', 'rbf', 'poly']
models_banknote = {}

for kernel in kernels:
    # Initialize and train the SVM
    svm_model = SVC(kernel=kernel, random_state=42)
    svm_model.fit(X_train_banknote, y_train_banknote)

    # Store model
    models_banknote[kernel] = svm_model

    # Make predictions
    y_pred_banknote = svm_model.predict(X_test_banknote)

    # Print classification report
    print(f"SVM with {kernel.upper()} Kernel <PES1UG23AM064>")
    print(classification_report(y_test_banknote, y_pred_banknote,
target_names=['Forged', 'Genuine', 'ExtraClass']))
    print("-" * 40 + "\n")

```

SVM with LINEAR Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
Forged	1.00	1.00	1.00	19
Genuine	1.00	0.95	0.98	21
ExtraClass	0.93	1.00	0.97	14
accuracy			0.98	54
macro avg	0.98	0.98	0.98	54
weighted avg	0.98	0.98	0.98	54

SVM with RBF Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
Forged	1.00	1.00	1.00	19
Genuine	0.63	0.90	0.75	21

ExtraClass	0.60	0.21	0.32	14
accuracy			0.76	54
macro avg	0.74	0.71	0.69	54
weighted avg	0.75	0.76	0.72	54

SVM with POLY Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
Forged	0.95	1.00	0.97	19
Genuine	0.65	0.95	0.77	21
ExtraClass	0.67	0.14	0.24	14

accuracy			0.76	54
macro avg	0.75	0.70	0.66	54
weighted avg	0.76	0.76	0.70	54

Step 2.3: Visualize Decision Boundaries

```
# --- Import Libraries ---
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import classification_report

# --- Step 1: Generate Synthetic 'Banknote' Data ---
X, y = make_classification(
    n_samples=500,
    n_features=4,           # similar to the real banknote dataset
    n_informative=3,
    n_redundant=0,
    n_classes=2,
    random_state=42
)

# --- Step 2: Scale the Features ---
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# --- Step 3: Reduce to 2D for Visualization ---
pca = PCA(n_components=2)
```

```

X_pca = pca.fit_transform(X_scaled)

# --- Step 4: Split the Data ---
X_train, X_test, y_train, y_test = train_test_split(X_pca, y,
test_size=0.3, random_state=42)

# --- Step 5: Define Function to Plot Decision Boundaries ---
def plot_decision_boundaries(X, y, model, title):
    x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
    y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
    xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                          np.linspace(y_min, y_max, 300))

    # Predict over the grid
    Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)

    # Plot
    plt.figure(figsize=(6, 5))
    plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
    plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.coolwarm,
edgecolors='k')
    plt.title(title)
    plt.xlabel("Principal Component 1")
    plt.ylabel("Principal Component 2")
    plt.show()

# --- Step 6: Train SVMs and Visualize ---
kernels = ['linear', 'rbf', 'poly']
models_banknote = {}

for kernel in kernels:
    model = SVC(kernel=kernel, random_state=42)
    model.fit(X_train, y_train)
    models_banknote[kernel] = model

    # Evaluate
    y_pred = model.predict(X_test)
    print(f"SVM with {kernel.upper()} Kernel <PES1UG23AM064>")
    print(classification_report(y_test, y_pred,
target_names=['Forged', 'Genuine']))
    print("-" * 40 + "\n")

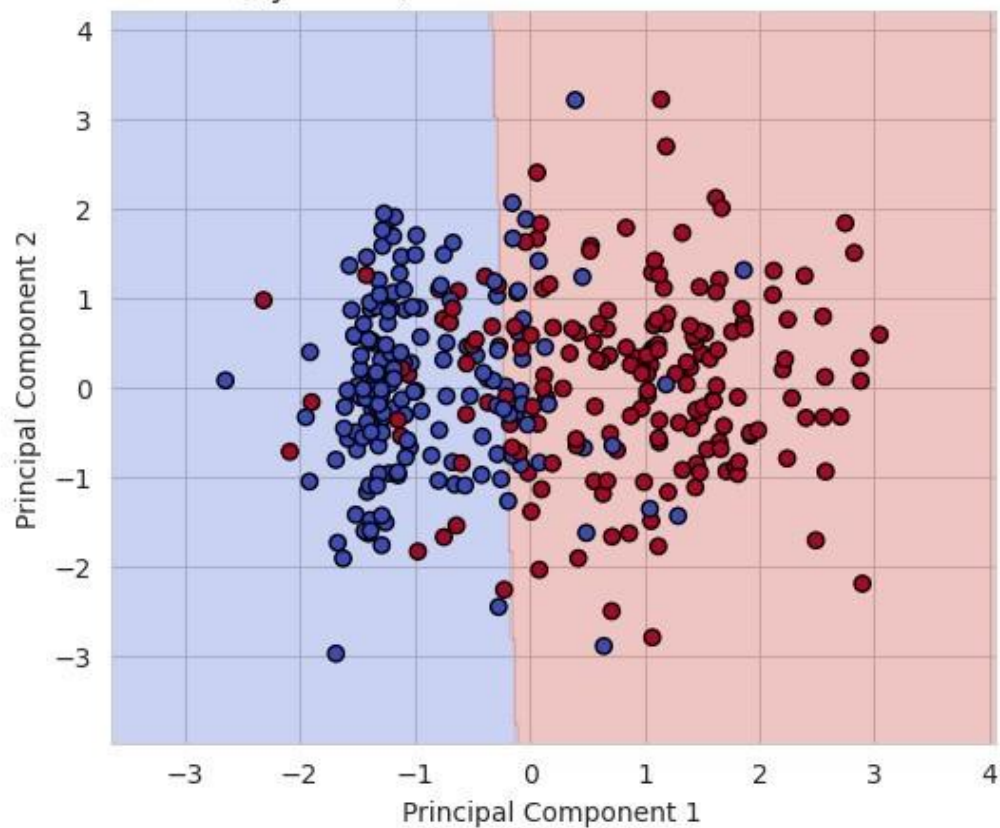
    # Plot decision boundary
    plot_decision_boundaries(
        X_train, y_train, model,
        title=f"Banknote Dataset (Synthetic) - SVM with
{kernel.upper()} Kernel <PES1UG23AM064>"
    )

```

SVM with LINEAR Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
Forged	0.84	0.78	0.81	76
Genuine	0.79	0.85	0.82	74
accuracy			0.81	150
macro avg	0.82	0.81	0.81	150
weighted avg	0.82	0.81	0.81	150

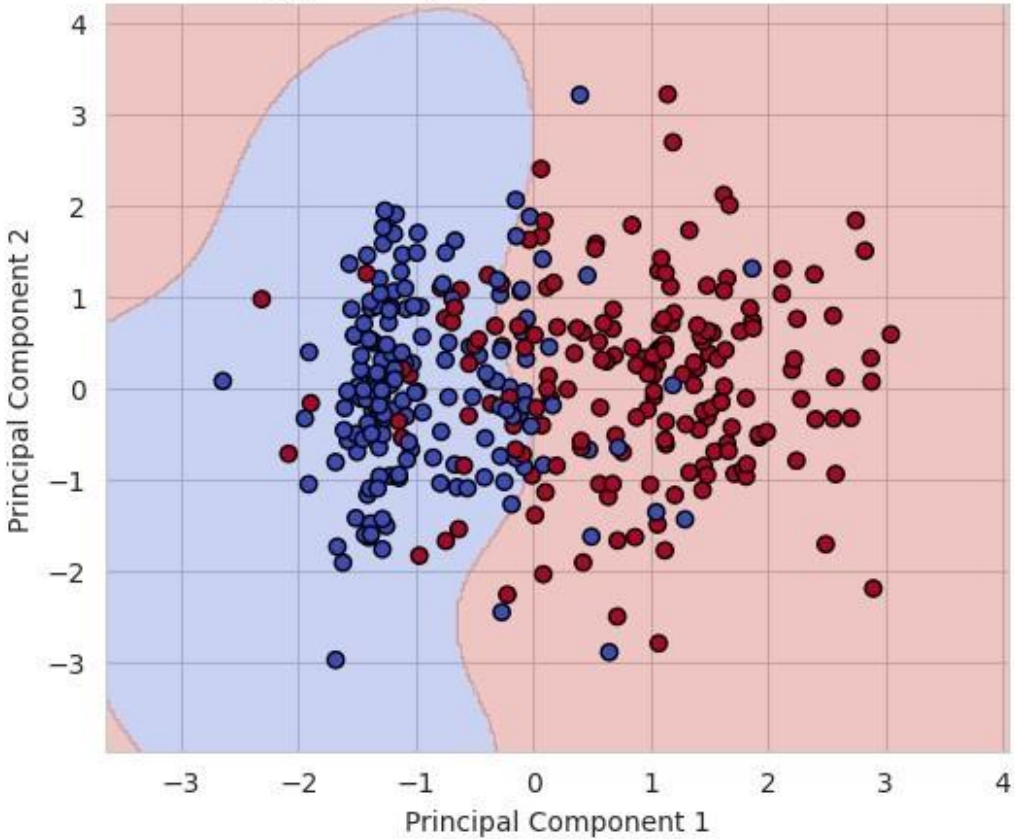
Banknote Dataset (Synthetic) - SVM with LINEAR Kernel <PES1UG23AM064>



SVM with RBF Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
Forged	0.84	0.78	0.81	76
Genuine	0.79	0.85	0.82	74
accuracy			0.81	150
macro avg	0.82	0.81	0.81	150
weighted avg	0.82	0.81	0.81	150

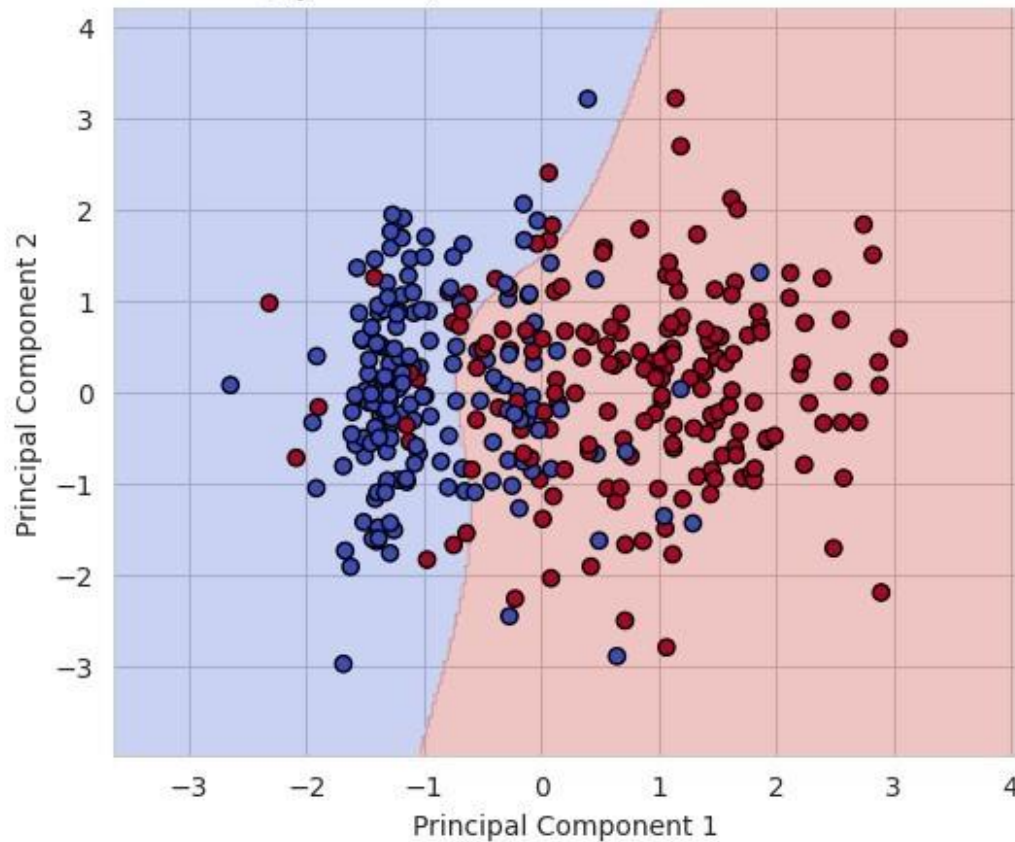
Banknote Dataset (Synthetic) - SVM with RBF Kernel <PES1UG23AM064>



SVM with POLY Kernel <PES1UG23AM064>

	precision	recall	f1-score	support
Forged	0.88	0.68	0.77	76
Genuine	0.74	0.91	0.81	74
accuracy			0.79	150
macro avg	0.81	0.79	0.79	150
weighted avg	0.81	0.79	0.79	150

Banknote Dataset (Synthetic) - SVM with POLY Kernel <PES1UG23AM064>



Most effective kernel: Linear Kernel – because the Banknote dataset is nearly linearly separable.

Polynomial kernel performs worse because it adds unnecessary complexity and overfits, while the data doesn't require curved decision boundaries (unlike the Moons dataset).

Most effective kernel: Linear Kernel – because the Banknote dataset is nearly linearly separable.

Polynomial kernel performs worse because it adds unnecessary complexity and overfits, while the data doesn't require curved decision boundaries (unlike the Moons dataset).

PART 3

4. Understanding the Hard and Soft Margins

Soft Margin (Small C value, e.g., 0.1): A smaller C value creates a "softer" margin, meaning the model is more tolerant of misclassifications. This results in a wider margin and can lead to better generalization, especially with noisy data.

Hard Margin (Large C value, e.g., 100): A larger C value creates a "harder" margin. The model will try to classify every data point correctly, resulting in a narrower margin. This can lead to overfitting if the data has outliers.

Let's create a dataset that is mostly linearly separable but has some noise, which is perfect for understanding the difference between hard and soft margins.

```
from sklearn.datasets import make_blobs

# Generate linearly separable data with some noise
X_linear, y_linear = make_blobs(n_samples=100, centers=2,
                                random_state=0, cluster_std=0.60)

# Add some outliers
outliers_X = np.array([[0.5, 2.5], [1.5, 0.5]])
outliers_y = np.array([1, 0])
X_linear = np.concatenate([X_linear, outliers_X])
y_linear = np.concatenate([y_linear, outliers_y])

# Split and scale the data
X_train_linear, X_test_linear, y_train_linear, y_test_linear =
train_test_split(
    X_linear, y_linear, test_size=0.3, random_state=42
)
scaler_linear = StandardScaler()
X_train_linear_scaled = scaler_linear.fit_transform(X_train_linear)
X_test_linear_scaled = scaler_linear.transform(X_test_linear)
```

Now, let's train two SVM models with a linear kernel: one with a small C (soft margin) and one with a large C (hard margin).

```
from sklearn.svm import SVC

# --- Soft Margin SVM (small C) ---
# C = 0.1 → allows some misclassifications → wider margin
svm_soft = SVC(kernel='linear', C=0.1, random_state=42)

# Fit the soft margin model
svm_soft.fit(X_train_linear_scaled, y_train_linear)

# Replace with your SRN
plot_decision_boundaries(
    X_train_linear_scaled, y_train_linear,
    svm_soft, title='Soft Margin SVM (C=0.1) <PES1UG23AM064>'
)

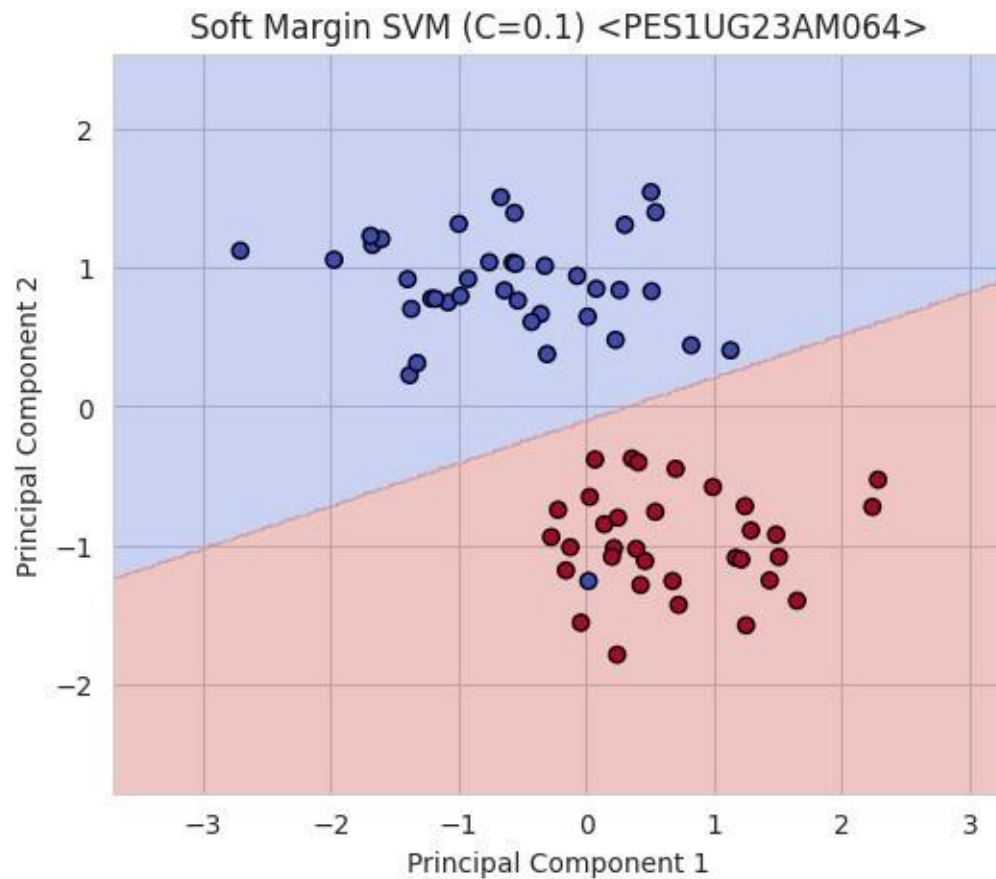
# --- Hard Margin SVM (large C) ---
# C = 100 → very strict on misclassifications → narrower margin
svm_hard = SVC(kernel='linear', C=100, random_state=42)
```

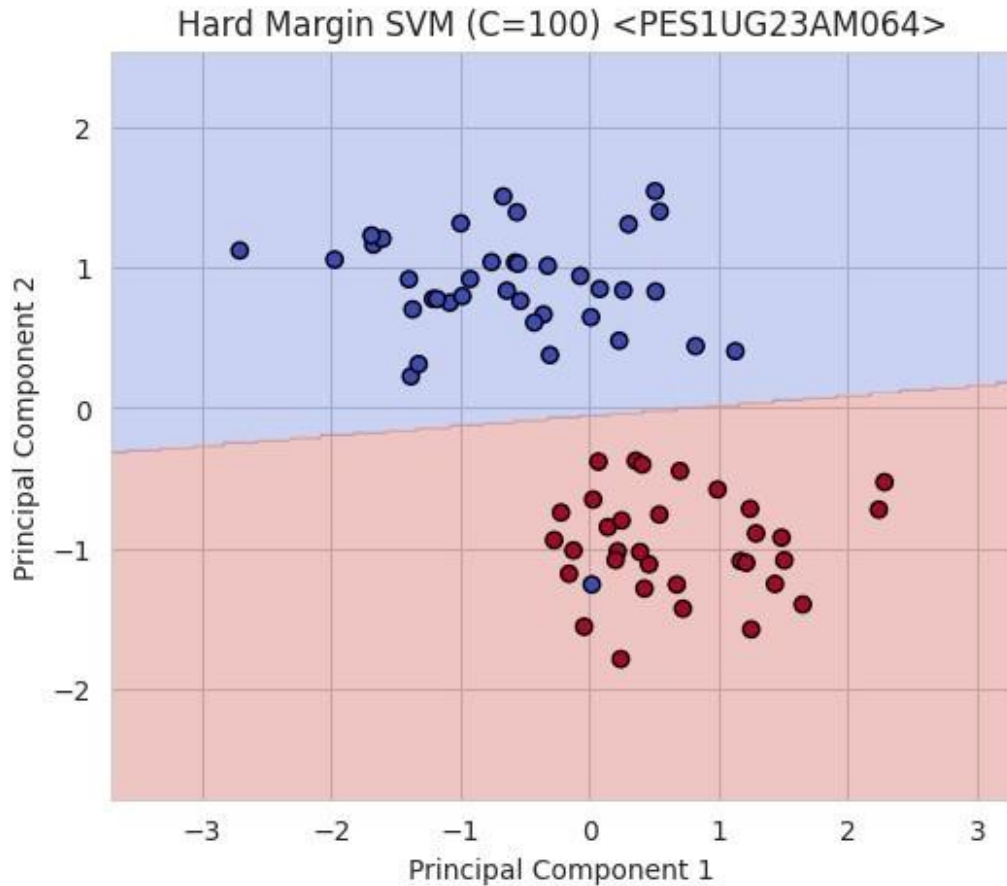
```

# Fit the hard margin model
svm_hard.fit(X_train_linear_scaled, y_train_linear)

# Replace with your SRN
plot_decision_boundaries(
    X_train_linear_scaled, y_train_linear,
    svm_hard, title='Hard Margin SVM (C=100) <PES1UG23AM064>'
)

```





Analysis Questions

1. Compare the two plots. Which model, the "Soft Margin" (C=0.1) or the "Hard Margin" (C=100), produces a wider margin?
2. Look closely at the "Soft Margin" (C=0.1) plot. You'll notice some points are either inside the margin or on the wrong side of the decision boundary. Why does the SVM allow these "mistakes"? What is the primary goal of this model?
3. Which of these two models do you think is more likely to be overfitting to the training data? Explain your reasoning.
4. Imagine you receive a new, unseen data point. Which model do you trust more to classify it correctly? Why? In a real-world scenario where data is often noisy, which value of C (low or high) would you generally prefer to start with?

Answers

1. **Wider Margin:** The **Soft Margin (C = 0.1)** produces a **wider margin** because it allows more flexibility and tolerates some misclassifications.
2. **Why SVM allows mistakes:** The **Soft Margin** model allows some points inside or on the wrong side of the margin to achieve a **better generalization**. Its goal is not

perfect accuracy on training data, but to **maximize the margin** and minimize overall classification error.

3. **More likely to overfit:** The **Hard Margin ($C = 100$)** model is more likely to **overfit** because it tries to classify every training point correctly, even noisy or outlier ones.
 4. **More trustworthy model for unseen data:** The **Soft Margin ($C = 0.1$)** model is more reliable for new, unseen data since it's less sensitive to noise and generalizes better. In real-world, noisy datasets, you'd generally prefer a **lower C** to start with.
-

5. Lab Summary and Conclusion

In this lab, you have:

- Trained SVM classifiers on three distinct datasets: one synthetic non-linear, one real-world binary, and one high-dimensional multi-class.
 - Implemented and compared three common kernels: Linear, RBF, and Polynomial.
 - Evaluated model performance using standard classification reports.
 - Visualized decision boundaries to understand how each kernel operates on different data distributions.
 - Understood hard and soft margins.
-